Bifrost documentation

Architecture overview

Bifrost is an optimistic bridge that leverages Cardano Stake Pools high decentralization level to secure the peg-ins and peg-outs from and to other UTxO blockchains like Bitcoin, Dogecoin and Litecoin. Because of the limited scripting capabilities of these blockchains, in recent years different bridging alternatives have been proposed. The current most known alternatives are FROST signatures of a small set of external nodes (Stacks), BitVM optimistic behaviour with 1-of-n honesty assumption with limited availability (Cardinal, Citrea) and Watchtower multisignature behaviour (Rosen Bridge).

Bifrost takes inspiration from all these solutions, but this time Cardano is used as a core component to guarantee the security and uncensorability of the user's actions.

It then becomes easier to connect Cardano, a UTXO blockchain with smart contracts, to other smart contract blockchains and Layer 2s, making Cardano the central component of a safe bridging process.

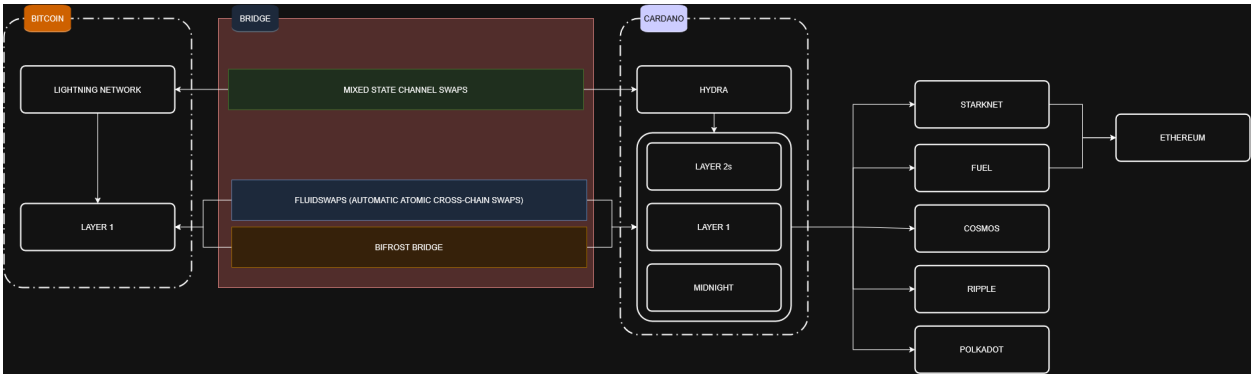


Figure 1: General bridge design

The Cardano SPOs collectively become the responsible custodians of bridged assets on the original blockchain. For example, SPOs keep and manage the locked BTC on the Bitcoin side, while its bridged version fBTC circulates freely on Cardano.

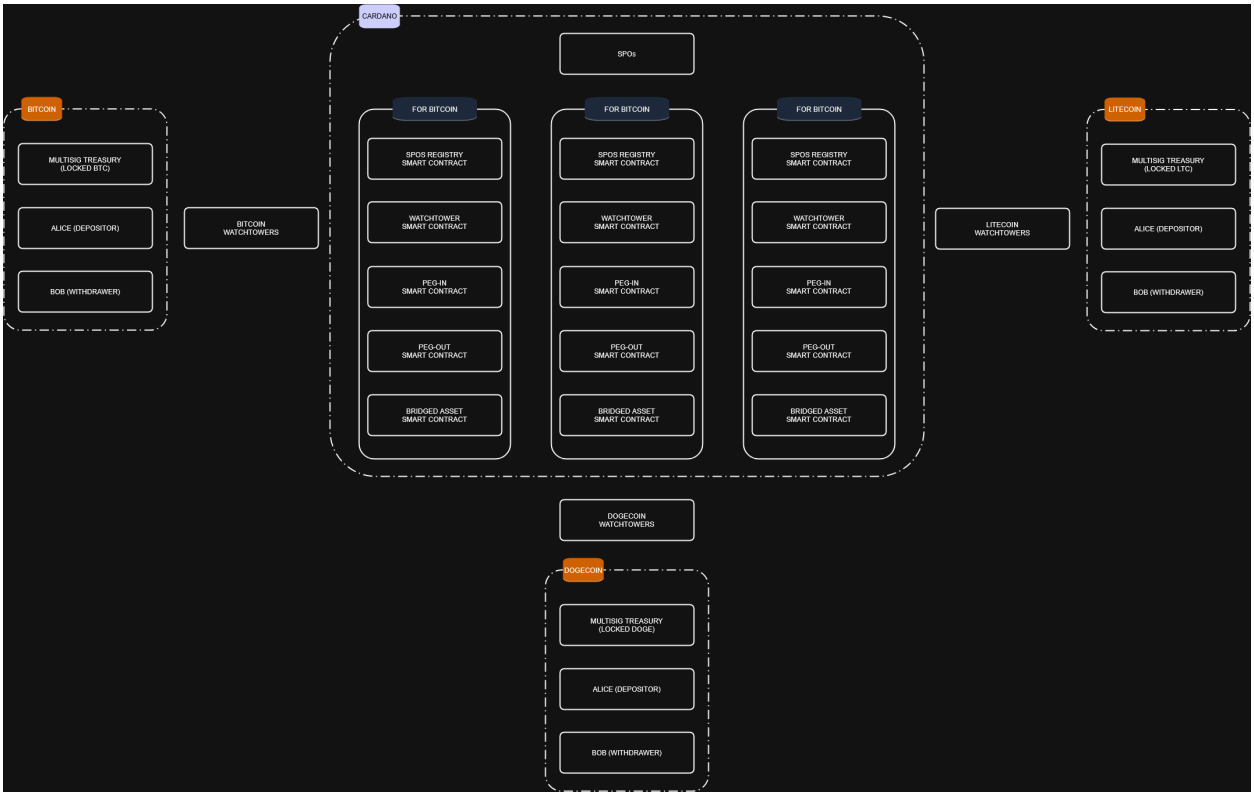
Bridge	Stacks Frost bridge	BitVM2	Rosen Bridge	Bifrost
Security assumption	Trust in small set of L2 nodes	At least 1 actor must honestly forget his private key	Trust in a set of nodes from a low marketcap blockchain	Weighted-majority of Cardano SPOs must behave honestly
Peg-in & Peg-out Availability	L2 nodes must be collaborative	Pre-chosen fixed set of operators must be collaborative	Majority of guards must be collaborative	Weighted-majority of Cardano SPOs must be collaborative

Bridge	Stacks Frost bridge	BitVM2	Rosen Bridge	Bifrost
Peg-in & Peg-out Granularity	Any amount	Fixed static amounts	Any amount	Any amount
Speed in good case	Minutes	Minutes	Minutes	1 Week
Speed in pessimistic case	Minutes	Weeks	Minutes	Weeks
Costs	Low	Medium	Low	Medium

Bifrost has been built to ensure security and availability, not speed or low costs. In fact, Bifrost operations may take up to 1 or more Cardano epochs (an epoch is currently equals to 5 days), as coordination and heavy operations must be executed in the correct order. The peg-ins and peg-outs also have to compensate for the work of all actors involved in Bifrost. Therefore Bifrost should be used to move big amounts of liquidity in and out of Cardano and not for intra-day retail/small business operations. Once big amounts of liquidity have been bridged to Cardano, for this type of smaller and frequent peg-ins and peg-outs it is possible to safely use services like FluidToken FluidSwaps, cutting costs and execution time without sacrificing security.

The security of Bifrost is guaranteed by SPOs participation: for a strong and reliable bridge, most of the top SPOs by delegation must participate in the protocol.

Components



Bifrost setup is made by the following components:

- **Cardano**: the destination blockchain where bridged assets can safely participate in DeFi activities.
- **Source blockchain**: the original blockchain that contains assets to bridge to Cardano, like Bitcoin, Dogecoin and Litecoin.
- **Depositors**: users that lock their assets on the source blockchain to mint them on Cardano.
- **Withdrawers**: users that burn their bridged assets on Cardano to unlock them on the proper source blockchain.
- **Cardano Stake Pool Operators (SPOs)**: Cardano nodes that have delegated stake by Cardano users and that participate in Cardano consensus, guaranteeing its security.
- **Multisig treasury**: a script address on the source blockchain that holds all the bridged assets and it's protected by a multisignature that only SPOs together can use. Each SPO has a weight equal to its delegation and a specific threshold of SPOs signature must be reached to spend/move the multisig treasury.
- **Watchtowers**: an open and always dynamic set of actors who have visibility on both Cardano and the source blockchain. They compete to post the most truthful source blockchain chain of blocks to the Binocular Oracle on Cardano. They also detect peg-in transactions on the source blockchain and post them as PegInRequest UTxOs on Cardano, and they relay SPO-signed Treasury Movement transactions from Cardano to the source blockchain. Anyone can become a Watchtower at any moment.

Bifrost logic is fully encapsulated in the following solutions:

- **SPOs program**: this code must run along with the usual SPO stack. It gives SPOs the ability to coordinate to sign Bitcoin transactions and the ability to see and interact with the needed Cardano smart contracts.
- **Watchtower program**: watchtowers run this software on top of source blockchain and Cardano nodes. It posts source blockchain block headers to the Binocular Oracle, detects peg-in transactions and posts PegInRequest UTxOs on Cardano, and relays SPO-signed Treasury Movement transactions to the source blockchain.
- Cardano smart contracts:
  - **spos_registry.ak**: SPOs that participate in Bifrost need to register here for the next upcoming epoch. The registry is an on-chain linked list ordered by SPOs edcs key and each node also contains the SPO secp key that will be used to sign source blockchain transactions.
  - **Binocular**: The watchtowers (anyone) post the best chain of blocks here, other watchtowers eventually challenge it by posting a better version and the winner gets rewarded by the end of the availability window.
  - **peg_in.ak**: watchtowers (or anyone) create PegInRequest UTxOs here by minting a PegInRequest NFT and providing a Binocular inclusion proof of the Bitcoin deposit transaction. The datum contains the raw Bitcoin peg-in transaction bytes. SPOs do not have direct access to Bitcoin chain state, so PegInRequest UTxOs serve as their trusted source of Bitcoin deposit data for constructing Treasury Movement transactions.
  - **peg_out.ak**: when a withdrawer wants to unlock the bridged assets on the proper source blockchain, he locks his bridged assets at this smart contract. The datum contains the source blockchain destination address where assets should be sent. SPOs read these UTxOs to include peg-out payments in the Treasury Movement transaction.
  - **treasury.ak**: stores the current Treasury FROST group public keys Y_{67} and Y_{51} and a Merkle Patricia Trie of completed peg-ins as a reference UTxO. The keys correspond to two FROST groups with thresholds ensuring any signing subset controls ≥67% and ≥51% of stake respectively. After each pair of DKGs, the current roster posts the new group public keys here, authenticated by a FROST group signature from the current roster. Depositors and validators read these to derive the current Treasury address. The completed peg-ins trie is updated each time fBTC is minted, preventing double minting. For the first epoch, the initial Treasury public keys are set during protocol bootstrap.
  - **treasury_movement.ak**: SPOs post signed source blockchain Treasury Movement transactions here. The datum contains the serialized signed transaction, the epoch number, and references to the PegInRequest and PegOut UTxOs it covers. Watchtowers monitor this contract and relay the signed transactions to the source blockchain.
  - **bridged_asset.ak**: minting and burning of bridged assets (e.g. fBTC). The depositor mints fBTC by spending the PegInRequest UTxO and providing: a Binocular inclusion proof of the confirmed Treasury Movement transaction, a reference to the corresponding `treasury_movement.ak` UTxO (to verify the confirmed transaction matches what SPOs signed and posted), a non-inclusion proof against the completed peg-ins Merkle Patricia Trie in `treasury.ak` (preventing double minting), their Bitcoin x-only public key, and a Schnorr signature proving ownership. The validator verifies the Binocular-confirmed txid matches the `treasury_movement.ak` datum (proving the confirmed transaction matches what was posted by the proto-

col's signing cascade), parses the raw TM transaction to verify the depositor's peg-in txid+vout appears as an input (proving the Treasury Movement actually swept the deposit), parses the raw peg-in transaction from the PegInRequest datum to extract the depositor_pubkey_hash from the OP_RETURN and the deposit amount, verifies HASH160(pubkey) matches the depositor_pubkey_hash, checks the signature via `verifySchnorrSecp256k1Signature`, verifies the peg-in is not already in the completed trie, and mints the correct amount of fBTC to whatever Cardano address the depositor specifies in the transaction outputs. The minting transaction also inserts the peg-in into the completed peg-ins trie in the Treasury UTxO. Anyone can burn fBTC for a peg-out by spending the PegOut UTxO and providing a Binocular inclusion proof of the Treasury Movement transaction that fulfilled the peg-out.

Components relationships

Watchtowers, who run the watchtower program, challenge each other to be the first to post the best source blockchain chain of valid blocks in the Binocular Oracle smart contract. The winner for each chain is rewarded with some ADA, proportionally for each valid block posted.

Depositors, who want to peg-in, send their source blockchain assets to a unique Taproot address with an OP_RETURN metadata marker identifying the transaction as a Bifrost peg-in. Watchtowers detect these transactions on the source blockchain, and once confirmed, create PegInRequest UTxOs on Cardano (peg_in.ak) by minting an NFT and providing an inclusion proof.

Withdrawers, who want to peg-out, lock their bridged assets (e.g. fBTC) along with a freshly minted NFT at peg_out.ak, specifying their source blockchain destination address in the datum.

SPOs, who register with their delegated stake to join the next epoch in spos_registry.ak, own both a unique edcs key and a secp key. The registration is accepted only if the SPO has a delegated stake bigger than a minimum threshold.

At the end of each epoch, the registered SPOs (that normally also include the old group) verify each other's delegated stake to ensure honesty and participate in a DKG ceremony to generate their new shared multisignature address.

The old SPOs group then constructs a Treasury Movement transaction on the source blockchain. All quorum levels construct the same **full** Treasury Movement transaction:

- Spends the current treasury UTxO, sending remaining funds to the new SPOs Treasury address.
- Collects (spends) all confirmed peg-in UTxOs, consolidating them into the treasury.
- Sends the correct amounts from the treasury to the source blockchain addresses that have correctly requested a peg-out.

The signing cascade tries higher quorum levels first for stronger security:

1. **67% quorum (Y_{67}, aspirational)**: SPOs sign via the Y_{67} script leaf in the Treasury Taproot tree. This proves the strongest security threshold on Bitcoin.
2. **51% quorum (Y_{51}, main line)**: SPOs sign via the Y_{51} key path — the cheapest spending path. This is the primary operating mode.

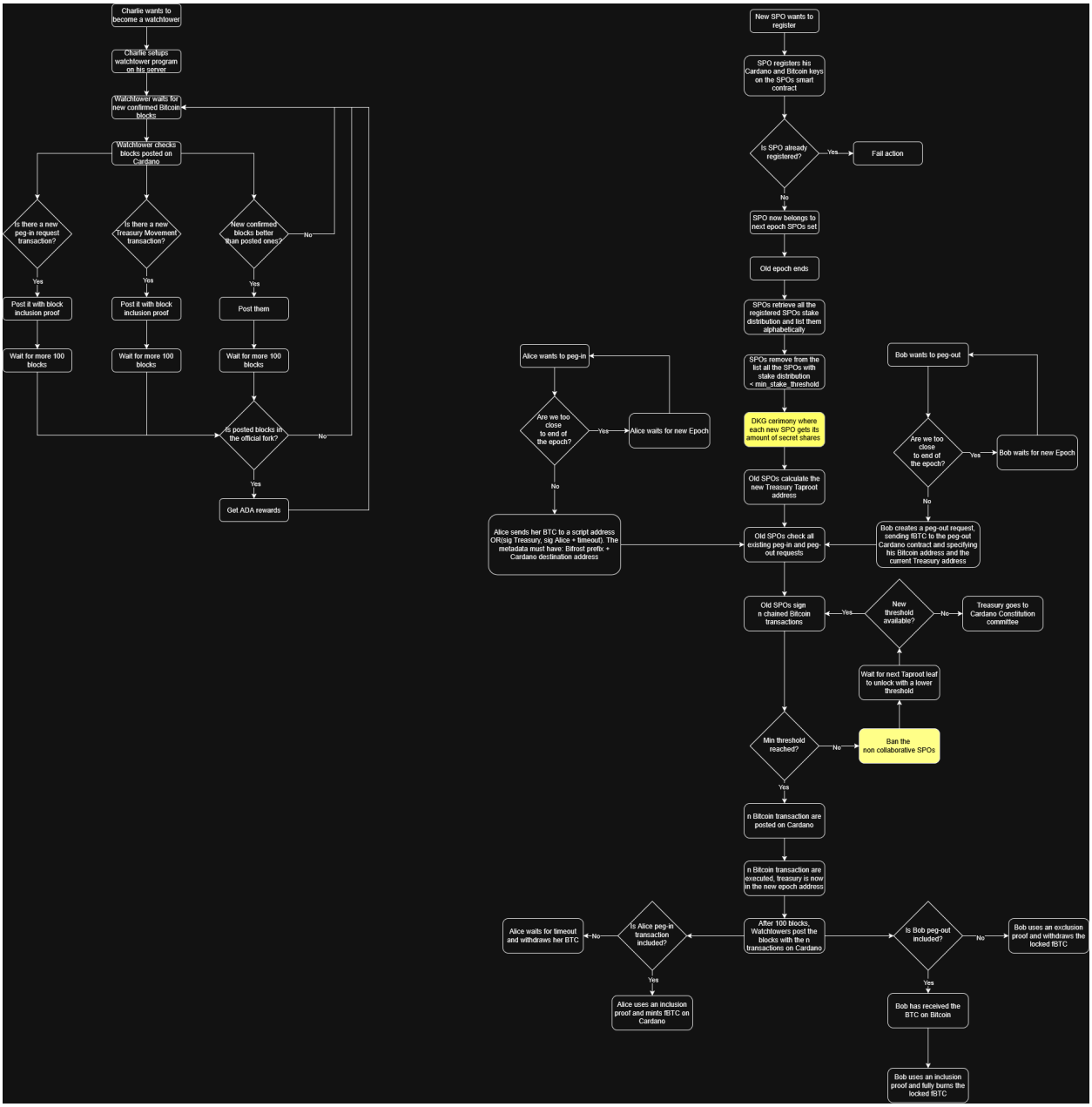


Figure 2: Bifrost Flow Chart

3. **Federation ($Y_{federation}$, emergency)**: if neither quorum is reached within the timeout, the federation signs via the $Y_{federation}$ script leaf with timelock.

If the resulting transaction would be too large, SPOs may split it into multiple transactions.

The SPOs sign this transaction using FROST group signing and post the serialized signed transaction to Cardano (treasury_movement.ak). Watchtowers monitor treasury_movement.ak, pick up the signed transaction, and broadcast it to the source blockchain network.

Once the Treasury Movement transaction is confirmed on the source blockchain, the bridging operations can be completed on Cardano:

- For peg-ins: the depositor spends the PegInRequest UTxO and provides a Binocular inclusion proof of the confirmed Treasury Movement transaction and a reference to the corresponding `treasury_movement.ak` UTxO — the validator verifies the confirmed txid matches the posted datum, proving the confirmed transaction matches what was posted by the protocol's signing cascade (not, e.g., a depositor timeout reclaim). The validator parses the raw TM transaction to verify the depositor's peg-in txid+vout appears as an input (proving the TM actually swept this deposit), and parses the raw peg-in transaction from the PegInRequest datum to extract the depositor_pubkey_hash and deposit amount. The depositor additionally provides a non-inclusion proof against the completed peg-ins Merkle Patricia Trie in the Treasury UTxO, their Bitcoin x-only public key, and a Schnorr signature proving ownership. This mints the corresponding fBTC to a Cardano address of the depositor's choice and inserts the peg-in into the completed peg-ins trie to prevent double minting.
- For peg-outs: anyone spends the PegOut UTxO and provides a Binocular inclusion proof of the Treasury Movement transaction that fulfilled the peg-out, burning the locked fBTC and retrieving the min_utxo ADA.

Peg-out completion is fully permissionless. Peg-in completion requires the depositor's action (Schnorr signature), which gives the depositor full control over the Cardano destination address.

Cardano and Bitcoin transaction flow

User peg-in flow

Let's use Bitcoin as example. A user who wants to move his BTC from Bitcoin to Cardano is called a depositor. These are the steps to execute a correct peg-in:

- Check the status of Bifrost: if the bridge is correctly operational and we are not too near the end of the current Cardano epoch, the peg-in can be done.
- Retrieve the current Treasury key Y_{51} from `treasury.ak` on Cardano (published there after each DKG).
- On Bitcoin, send the amount of BTC to peg-in to a Taproot address derived from Y_{51}, the federation fallback script, and the depositor's timeout refund script (see **Taproot address construction** below). The address has three spending paths: the Y_{51} key path (for SPO sweep — main line), a $Y_{federation}$ script leaf (for federation emergency sweep after timeout), and a script leaf allowing the depositor to reclaim after ~30 days. The transaction must include an OP_RETURN output containing: `"BFR" || depositor_pubkey_hash`

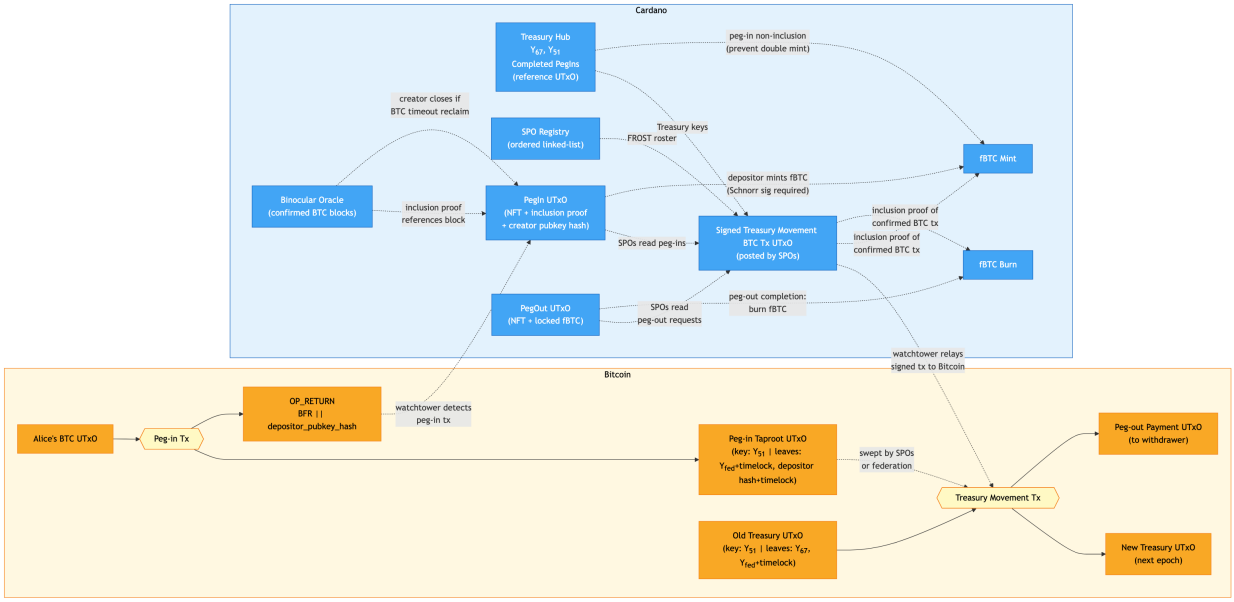


Figure 3: Bifrost UTxO Flow

`(20 bytes)` (23 bytes total). The `depositor_pubkey_hash` is HASH160 of the depositor's Bitcoin x-only public key and is needed by SPOs to reconstruct the Taproot address and compute the tweak for key-path signing.

- Wait for watchtowers to detect the Bitcoin transaction, post the corresponding Bitcoin block to the Binocular Oracle, and create a PegInRequest UTxO on Cardano (peg_in.ak) by minting a PegInRequest NFT and providing a transaction inclusion proof.
- Wait for the SPOs to include this peg-in in the Treasury Movement transaction at the next epoch boundary. The SPOs sign this transaction with FROST and post it to Cardano (treasury_movement.ak). Watchtowers then relay the signed transaction to Bitcoin.
- Once the Treasury Movement transaction is confirmed on Bitcoin, the depositor completes the peg-in on Cardano by spending the PegInRequest UTxO and providing: a Binocular inclusion proof of the confirmed Treasury Movement transaction, a reference to the corresponding `treasury_movement.ak` UTxO (the validator verifies the confirmed txid matches the posted datum, proving the confirmed transaction matches what was posted by the protocol's signing cascade), a non-inclusion proof against the completed peg-ins Merkle Patricia Trie in the Treasury UTxO (preventing double minting), their Bitcoin x-only public key, and a Schnorr signature proving ownership. The validator parses the raw TM transaction to verify the depositor's peg-in txid+vout appears as an input (confirming the Treasury Movement actually swept this deposit), and parses the raw peg-in transaction from the PegInRequest datum to extract the depositor_pubkey_hash and deposit amount (this is the only point where the peg-in transaction is parsed on-chain). This mints the correct amount of fBTC to whatever Cardano address the depositor chooses and inserts the peg-in into the completed peg-ins trie in the Treasury UTxO.
- If the peg-in was not included in the Treasury Movement transaction (e.g., it arrived too late in the epoch), it rolls over to the next epoch. If the Treasury key has rotated and the peg-in can no longer be swept, the depositor uses the ~30-day timeout spending path to reclaim their BTC and can retry with the new Treasury address.

- **PegInRequest closure**: A PegInRequest UTxO can be closed (NFT burned, min_utxo ADA reclaimed by the creator) under two conditions:
  - **After depositor timeout reclaim**: the creator provides a Binocular inclusion proof of a confirmed Bitcoin transaction that spends the peg-in txid+vout via the **depositor refund script leaf** (not the federation leaf, not the key path). The on-chain validator parses the Bitcoin transaction witness to verify it is a script-path spend using the depositor refund script specifically, not a key-path spend (which would be an SPO sweep) or a federation script-path spend (which would also be a legitimate sweep). This ensures closure cannot grief a depositor whose funds were legitimately swept by either SPOs or the federation.
  - **Duplicate PegInRequest**: the creator provides a **trie inclusion proof** showing the peg-in is already in the completed peg-ins Merkle Patricia Trie in the Treasury UTxO. This means fBTC was already minted via another PegInRequest for the same deposit, so this one is redundant.

Taproot address construction

The Treasury address and peg-in addresses use different Taproot trees following BIP341 [4]. Both use Y_{51} as the key-path internal key, making the 51% FROST threshold the primary ("main line") operating mode. The 67% threshold appears as a script leaf in the Treasury tree for aspirational stronger security, and the federation appears as a timelock-gated fallback in both trees.

Keys

- Y_{67} and Y_{51} are FROST group public keys produced by **separate DKGs** with thresholds ensuring any signing subset controls ≥67% and ≥51% of delegated stake respectively. Both are stored in `treasury.ak`.
- $Y_{federation}$ is a known protocol parameter — a public key controlled by a federation of trusted entities, used only as a last-resort spending path.

Treasury Taproot tree The Treasury address (holding consolidated funds) uses Y_{51} as the key-path internal key, with an aspirational stronger path and an emergency fallback:

Path	Key	Condition	Use case
Key path	Y_{51}	Immediate	Normal operation (main line): full TM
Script leaf 1	Y_{67}	Immediate	Aspirational: full TM with strongest security proof
Script leaf 2	$Y_{federation}$	After timeout	Emergency fallback: full TM

When 67% quorum is available, SPOs prefer Y_{67} (script leaf 1) to prove the stronger security threshold on-chain on Bitcoin, even though it costs slightly more than the key path. When 67% is not available, they fall back to Y_{51} key path (main line, cheapest).

Script leaf 1 (Y_{67} aspirational):

```
<Y_67> OP_CHECKSIG
```

Script leaf 2 (federation rescue):

```
<timeout_federation> OP_CHECKSEQUENCEVERIFY OP_DROP <Y_federation> OP_CHECKSIG
```

Merkle tree (2 leaves):

```
     root
    /    \
  Y_67  Y_federation
```

Treasury output key: $Q_{treasury} = Y_{51} + \text{tagged_hash}(\text{"TapTweak"}, Y_{51} \| \text{merkle_root}) \cdot G$

This address changes each epoch after DKG, since Y_{67} and Y_{51} are regenerated.

When 67% quorum is available, SPOs spend the treasury via the Y_{67} script leaf — proving the stronger security threshold on Bitcoin at a slightly higher cost. When only 51% quorum is available, SPOs use the Y_{51} key path — a single 64-byte Schnorr signature with no script reveal, the cheapest spending path. In emergency (federation), the $Y_{federation}$ script path with timelock is used.

Peg-in Taproot tree The peg-in address uses Y_{51} as the key-path internal key (for SPO sweep — main line), with a federation emergency sweep leaf and a depositor refund leaf:

Path	Key	Condition	Use case
Key path	Y_{51}	Immediate	SPO sweep (main line)
Script leaf 1	$Y_{federation}$	After timeout	Federation emergency sweep
Script leaf 2	Depositor	After ~30 days (4320 blocks)	Depositor self-refund

Script leaf 1 (federation emergency sweep):

```
<timeout_federation> OP_CHECKSEQUENCEVERIFY OP_DROP <Y_federation> OP_CHECKSIG
```

Script leaf 2 (depositor refund, P2PKH-style):

```
OP_DUP OP_HASH160 <depositor_pubkey_hash> OP_EQUALVERIFY OP_CHECKSIGVERIFY <4320> OP_CHEC
```

`depositor_pubkey_hash` is HASH160 of the depositor's Bitcoin x-only public key (20 bytes). 4320 blocks ≈ 30 days. At spend time, the depositor provides their full x-only pubkey in the witness; the script verifies the hash matches before checking the signature.

Merkle tree (2 leaves):

```
      root
     /    \
  Y_fed   depositor_refund
```

The peg-in output key Q is:

$Q = Y_{51} + \text{tagged_hash}(\text{"TapTweak"}, Y_{51} \| \text{merkle_root}) \cdot G$

Where:

- Y_{51} is the internal key (51% FROST group x-only public key, from `treasury.ak`).
- The script tree contains two leaves (federation sweep and depositor refund), so merkle_root is the hash of both leaf hashes.
- leaf_hash $=$ tagged_hash(”TapLeaf”, 0xc0$\|$compact_size(script_len)$\|$script)
- tagged_hash(tag, msg) $=$ SHA256(SHA256(tag)$\|$SHA256(tag)$\|$msg)
- G is the secp256k1 generator point.

The resulting Bitcoin address is `bc1p<bech32m(Q)>`.

To reconstruct Q, all components are available: Y_{51} and $Y_{federation}$ from `treasury.ak` and the depositor’s pubkey hash from the OP_RETURN (propagated via the PegInRequest datum). Both scripts are fully determined by these parameters — no secret information is needed.

Spending paths and Treasury Movement variants All quorum levels construct **full** Treasury Movement transactions (sweeping peg-in UTxOs, fulfilling peg-outs, and moving the treasury). The signing cascade tries higher quorums first:

Script path on Treasury, key path on peg-in inputs (67% quorum — aspirational):

SPOs collect all confirmed PegInRequest and PegOut UTxOs from Cardano and construct a full Treasury Movement transaction. They spend the treasury UTxO via the Y_{67} script leaf (revealing the script and control block) to prove the stronger 67% security threshold on Bitcoin. Peg-in UTxOs are spent via key path (Y_{51}) — a single 64-byte FROST Schnorr signature per peg-in input. To sign peg-in inputs, SPOs compute the tweaked private key: $d = y_{51} +$ tagged_hash(”TapTweak”, $Y_{51}\|$merkle_root), where y_{51} is the FROST group private key (held as shares). Computing the merkle_root requires the depositor’s pubkey hash (for the refund leaf) and $Y_{federation}$ (for the federation leaf) — both available from the PegInRequest datum and protocol parameters.

Key path on Treasury, key path on peg-in inputs (51% quorum — main line):

If SPOs cannot collect enough partial signatures for the 67% threshold, they sign via the Y_{51} key path on all inputs — the cheapest spending path. The same full Treasury Movement transaction (peg-ins + peg-outs + treasury move) is constructed.

Script path on Treasury, script path on peg-in inputs (federation — emergency):

If neither 67% nor 51% quorum is reached within the timeout, the federation signs the same full Treasury Movement transaction using $Y_{federation}$ (script path with CSV timelock on all inputs).

Script path on peg-in only (depositor refund):

After ~30 days (4320 blocks), the depositor reveals the depositor refund script and control block to reclaim their BTC. This protects depositors if the bridge fails to process their peg-in (e.g., Treasury key rotated before sweep).

Taproot address verification Plutus V3 does not expose secp256k1 point arithmetic builtins (only `verifySchnorrSecp256k1Signature` and `verifyEcdsaSecp256k1Signature`), so `peg_in.ak` **cannot** reconstruct Q from Y_{51}, $Y_{federation}$, and the depositor’s script on-chain.

Instead, Taproot address correctness is verified **off-chain by SPOs**: before including a peg-in in the Treasury Movement transaction, each SPO independently reconstructs the expected peg-in Taproot address from Y_{51}, $Y_{federation}$, and the depositor's pubkey hash (read from the PegInRequest datum), and verifies it matches the Bitcoin transaction output. SPOs will not sign a Treasury Movement transaction that spends UTxOs they cannot actually spend.

This design is safe because:

- **No fund risk**: if a PegInRequest references an incorrectly constructed Taproot address, SPOs simply skip it. The depositor reclaims via the timeout path.
- **No theft risk**: fBTC is only minted after the Treasury Movement transaction (which sweeps the peg-in) is confirmed on Bitcoin. A fake PegInRequest that SPOs skip will never lead to fBTC minting.
- **Griefing cost**: creating a fake PegInRequest costs the attacker the NFT minting fee and min_utxo ADA, with no benefit.

User peg-out flow

Let's use Bitcoin as example. A user who wants to move his BTC from Cardano to Bitcoin is called a withdrawer. These are the steps to execute a correct peg-out:

- Check the status of Bifrost: if the bridge is correctly operational and we are not too near the end of the current Cardano epoch, the peg-out can be done.
- On Cardano, lock the correct amount of fBTC plus MIN_ADA at the peg_out.ak spend script, minting a unique NFT. The datum contains the Bitcoin destination address where BTC should be sent.
- Wait for the SPOs to include this peg-out in the Treasury Movement transaction at the next epoch boundary. The SPOs sign this transaction with FROST and post it to Cardano (treasury_movement.ak). Watchtowers then relay the signed transaction to Bitcoin. At this point, the withdrawer has received BTC at their specified Bitcoin address.
- Once the Treasury Movement transaction is confirmed on Bitcoin (100 Bitcoin blocks for Binocular confirmation), anyone can complete the peg-out on Cardano by providing a Binocular inclusion proof of the Treasury Movement transaction. This burns the locked fBTC and the peg-out NFT, returning the MIN_ADA to the withdrawer.
- If for unexpected reasons the Treasury Movement transaction did not include the peg-out payment, the withdrawer can use a Binocular exclusion proof to unlock their fBTC and try again in the next epoch.

Guaranteeing censor-resistant peg-ins and peg-outs

The main axiom is: When the user uses any bridge, he is already fully trusting the source (ex. Bitcoin) and the destination (ex. Cardano). Every additional component that the bridge uses and that it can't be under direct control of the user is an additional trust assumption.

Bifrost is truly trustless only if it doesn't necessarily add new trust assumptions. As long as the Cardano SPOs and the watchtowers are collaborative, each peg-in or peg-out is permissionless: no actor exists who can decide if the user is permitted to move his assets between the blockchains.

Therefore, the potential additional trust assumptions in Bifrost are the Cardano SPOs and the watchtowers:

- Even if the user becomes a Cardano SPO, he would be just a small part of the total weight-based set of SPOs. Luckily, the strong majority of the SPOs are always incentivized in behaving correctly and on time, like they do when they participate in block-production consensus on Cardano. In fact, the security of Bifrost directly impacts their revenue model: more assets moved with Bifrost imply more Cardano transactions and an increase of the ADA price caused by the bigger demand to execute these transactions. Cardano SPOs want the bridge to work well because their revenue stream strongly depends on it.
- Watchtowers are an "always open" set of nodes that challenge each other to post on Cardano the best chain of blocks from the source blockchains (ex. from Bitcoin), and also detect and post peg-in requests on Cardano. While the watchtowers earn rewards for doing this job, they could potentially collude and stop posting blocks or peg-in requests, halting the bridge for an unbounded timeframe. In this case the user who wants to peg-in or peg-out can spin up a watchtower himself and post the source blockchain blocks starting from the latest confirmed ones, and create their own PegInRequest UTxOs on Cardano. Because every user is able to become a watchtower at any time, there will be a safe challenge among them to post the correct chain of blocks, resuming the Bifrost operations even in case of collusion. The completion of peg-outs (burning fBTC) is fully permissionless: anyone can submit the required Binocular inclusion proofs to finalize. For peg-ins, the depositor completes the minting themselves by providing a Binocular inclusion proof and a Schnorr signature with their Bitcoin key, choosing their Cardano destination address at mint time. No third party can censor or redirect a depositor's fBTC.

Rollout Phases

Bifrost supports a phased rollout from federated to fully decentralized operation:

Phase 1 — Federation Launch: The bridge launches with the federation as the only signing entity. SPOs begin registering. The federation key is the $Y_{federation}$ used in the Taproot fall-back path. During this phase, all Treasury Movement transactions are signed via the federation script path with timelock.

Phase 2 — 51% SPO Participation: Once sufficient SPOs have registered and completed DKG, the 51% FROST threshold becomes operational. SPOs sign via key path (Y_{51}), and the federation becomes an emergency-only fallback. This is the "main line" operating mode — the protocol's primary steady-state.

Phase 3 — 67% SPO Participation (aspirational): As more SPOs join, the bridge achieves the aspirational 67% participation level. This doesn't change the signing key (still Y_{51} key path available) but provides stronger security: any signing subset now controls at least 67% of delegated stake, making attacks significantly more expensive. When 67% quorum is available, SPOs prefer to sign via the Y_{67} script leaf to prove the stronger security threshold on-chain on Bitcoin.

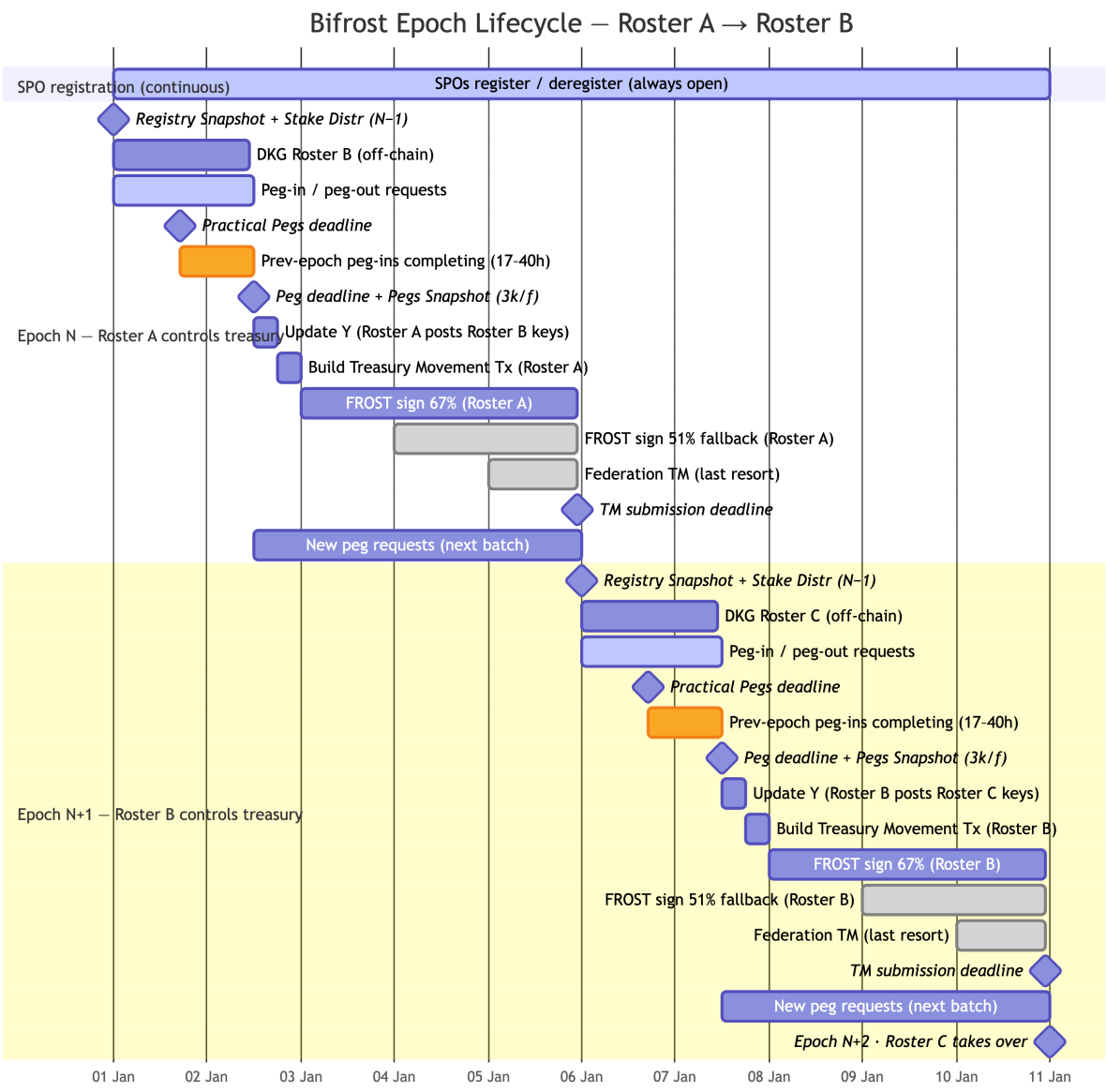


Figure 4: Epoch lifecycle Gantt diagram

Flow of Bitcoin over epochs, ceremonies

The diagram above shows two consecutive Cardano epochs with roster handoff from Roster A to Roster B. SPO registration and deregistration is continuous — a registry snapshot is taken at each epoch boundary along with the stake distribution from epoch N−1 (which will become N−2 when the new roster operates). Within each epoch the following phases occur:

1. **Registry Snapshot + Stake Distribution** — at the epoch boundary, the candidate set is locked and stake weights are read from the previous epoch's distribution.
2. **Peg-in / peg-out requests open** — users submit bridging requests during the first ~36 hours of the epoch.
3. **DKG** (new roster, off-chain) — the incoming roster runs distributed key generation to produce group keys Y_{67} and Y_{51}, running concurrently with the request window.
4. **Previous-epoch peg-in completion** — peg-ins from the prior epoch's Treasury Movement complete as Bitcoin confirmations arrive (17–40 hours after epoch start).
5. **Peg deadline + Pegs Snapshot** — at the Cardano stability window (3k/f), all bridging requests are frozen for inclusion in the Treasury Movement.
6. **Update Y** — the current roster publishes the new roster's group public keys to `treasury.ak`.
7. **Build Treasury Movement Tx** — the current roster constructs the Bitcoin transaction that sweeps peg-in UTxOs, fulfils peg-out payments, and moves the treasury to the new Taproot address.
8. **FROST signing cascade** — the current roster attempts threshold signing with overlapping quorum levels: 67% signing starts first, 51% fallback begins ~24 hours later, and the federation last-resort begins ~24 hours after that. The first to succeed wins. All quorum levels construct the same full Treasury Movement transaction (peg-ins + peg-outs + treasury move).
9. **TM submission deadline** — the signed transaction must be posted to `treasury_movement.ak` before the epoch ends.
10. **New peg requests** — after the pegs snapshot, new requests accumulate for the next epoch's batch.

Realistic epoch timeline (happy path)

The epoch lifecycle above shows generous time windows for the signing cascade (67% → 51% → federation). In the happy path, when 67% quorum is available, the epoch proceeds much faster:

- **DKG**: ~5 minutes (off-chain, SPOs communicate via `bifrost_url` endpoints).
- **FROST 67% signing**: ~1 minute per Treasury Movement transaction.
- **Multiple TM batches**: the roster processes peg requests in multiple batches throughout the epoch, each cycling through build → sign → broadcast → Bitcoin confirmation.

The bottleneck is Bitcoin confirmation: each Treasury Movement requires ~100 Bitcoin blocks (~16.7 hours) for Binocular to promote the containing block to `confirmed` state. With a 5-day Cardano epoch, 4–5 TM batches fit sequentially, each handling its own set of peg-in sweeps and peg-out fulfillments. The final TM of the epoch moves the treasury to the new roster's Taproot address.

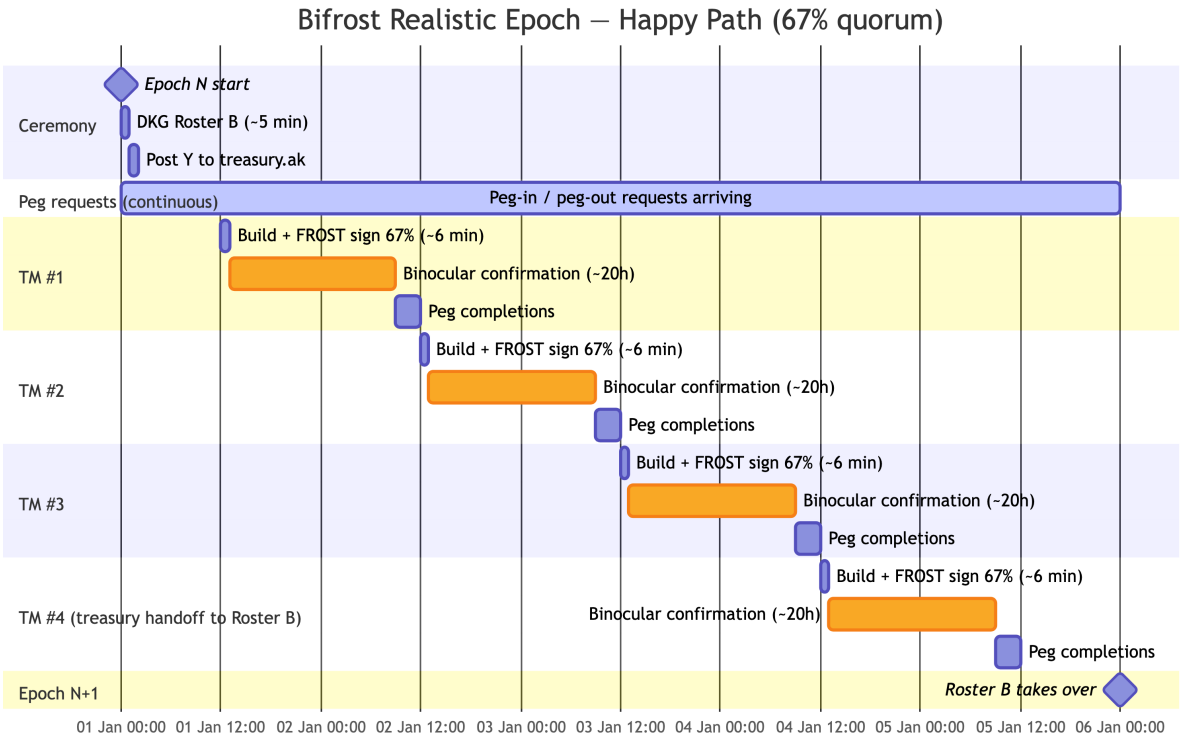


Figure 5: Realistic epoch lifecycle

Flow of SPOs on Cardano

todo

How to join Bifrost as SPO

todo

Why Mithril is not necessary

todo

Why Frost

todo

SPO Program

It's the program that Cardano SPOs must run and it allows signature aggregation. Being based on the FROST protocol requires: 1. registration of SPOs to participate in the protocol 2. formation of a roster of Cardano SPOs and distributed key generation (every epoch) 3. group signing. We describe each in detail.

SPO Registration

1. Overview Before participating in Bifrost, each SPO must complete a **one-time registration** that binds their Cardano pool identity to a long-term Bifrost identity key, and post a **security deposit**. This registration uses the SPO's cold key exactly once, after which all protocol operations use the Bifrost identity key. This design keeps cold keys offline except for initial registration and revocation.

2. Keys

2.1 SPO Identity (Cardano Layer)

- `pool_id`: unique stake pool identifier, derived as `pool_id = blake2b_224(cold_vkey)`.
- `cold_vkey` / `cold_skey`: long-term Ed25519 keypair. Used **only** for initial Bifrost registration and revocation. Must be kept on an air-gapped offline machine per Cardano security guidelines.

2.2 Bifrost Identity

- `bifrost_id_pk` / `bifrost_id_sk`: long-term Secp256k1 identity keypair for Bifrost protocol operations.
- Self-generated by the SPO.
- Used for roster participation, DKG coordination, and encryption of DKG shares (via ECDH).

2.3 Bifrost URL

- `bifrost_url`: endpoint URL where the SPO publishes DKG data and receives protocol messages.

3. On-Chain Objects

3.1 Bifrost Membership Token

- **Minting Policy**: `BifrostMembershipPolicy`
- **TokenName**: `pool_id`
- Exactly **one token per SPO** (enforced by minting policy).
- The token serves as the on-chain badge of Bifrost participation.

3.2 Registry Linked-List All registered SPOs are tracked using an **on-chain ordered linked-list**. Each node in the list represents a registered SPO and is stored as an individual UTxO at the registry script address. The list is ordered by SPO key, ensuring uniqueness and enabling efficient insertion and removal.

- **Node Value**: Bifrost Membership Token + minimum deposit amount in ADA.
- **Node Datum**:

```
{ key               :: ByteArray       -- SPO key (ordering key)
, next              :: ByteArray | Null -- key of the next node, or null for the tail
, data              ::
    { bifrost_id_pk :: ByteArray
    , bifrost_url   :: ByteArray
    }
}
```

Operations: - **Prepend/Insert**: A new node is inserted in sorted order by verifying it is correctly positioned between its neighbors. Corresponds to `ordered.prepend` in the on-chain code. - **Remove**: A node is removed by relinking its neighbors. Corresponds to `ordered.remove` in the on-chain code.

Spending Conditions: Each node UTxO can be spent by either: 1. **SPO withdrawal**: via `bifrost_id_sk` Secp256k1 signature, valid only at epoch boundary (enforced via Cardano validity intervals). 2. **Roster banning**: via FROST group signature from the current roster, for banning misbehaving participants.

The membership script uses `verifySchnorrSecp256k1Signature` for both conditions.

The on-chain linked-list implementation uses the `aiken_design_patterns/linked_list/ordered` module [5].

4. Registration Message and Signature The SPO's cold key signs a message binding their Bifrost identity:

```
"bifrost-spo" || bifrost_id_pk || bifrost_url
```

Where: - `"bifrost-spo"` is a 10-byte ASCII domain separator. - `bifrost_id_pk` is the 33-byte compressed Secp256k1 public key. - `bifrost_url` is the variable-length URL encoded as UTF-8 bytes.

5. Registration Transaction A **registration tx** performs the following:

1. **Redeemer**: contains `cold_vkey`, `cold_sig`, `prepended_node_output_index`, and `anchor_node_output_index`.
2. **Input**: Anchor node UTxO from the linked-list (the node after which the new node will be inserted).
3. **Mint**: exactly one Bifrost Membership Token with `TokenName = pool_id`.
4. **Outputs**:
   - New linked-list node UTxO at registry script address with:
     - Bifrost Membership Token + minimum deposit in ADA
     - Datum containing `bifrost_id_pk`, `bifrost_url`, and linked-list pointers (correctly ordered between neighbors)
   - Updated anchor node UTxO with its `next` pointer updated to reference the new node

6. On-Chain Verification The minting policy verifies:

1. `pool_id == blake2b_224(cold_vkey)` — proves the cold key owns this pool.
2. `verifyEd25519Signature(cold_vkey, "bifrost-spo" || bifrost_id_pk || bifrost_url, cold_sig)` — proves cold key authorized this Bifrost identity binding.
3. Exactly one token minted with `TokenName = pool_id`.
4. Output datum matches the signed message content.
5. **Linked-list ordering**: verifies the new node is correctly positioned between its neighbors (the new key is greater than the anchor's key and less than the anchor's previous `next` key), preventing duplicate registration.
6. **Linked-list state transition**: verifies the anchor node's `next` pointer is correctly updated to reference the new node.
7. **Security deposit**: verifies the Membership UTxO contains sufficient ADA.

7. Revocation An SPO's membership can end through voluntary revocation or roster-initiated banning.

7.1 Voluntary Revocation The SPO's cold key signs an explicit revocation message:

```
"bifrost-revoke" || pool_id
```

Where: - `"bifrost-revoke"` is a 14-byte ASCII domain separator. - `pool_id` is the 28-byte pool identifier.

Transaction: 1. **Redeemer**: contains `cold_vkey`, `cold_sig`, `removed_node_input_index`, and `anchor_node_input_index`. 2. **Validity interval**: must fall within the epoch boundary window. 3. Spends the Membership UTxO (returning the security deposit to the SPO). 4. Burns the Bifrost Membership Token. 5. Removes the node from the linked-list by updating the anchor node's `next` pointer to skip the removed node.

On-chain verification: 1. `pool_id == blake2b_224(cold_vkey)` — proves the cold key owns this pool. 2. `verifyEd25519Signature(cold_vkey, "bifrost-revoke" || pool_id, cold_sig)` — proves cold key authorized revocation. 3. **Validity interval**: transaction validity falls within the epoch boundary window. 4. Exactly one token burned with `TokenName = pool_id`. 5. **Linked-list removal**: verifies the anchor node's `next` pointer is correctly updated to skip the removed node, maintaining list ordering. 6. Security deposit returned to SPO.

After exit, the SPO may re-register with a new Bifrost identity.

7.2 Banning (Exponential Timeout) The protocol supports **temporary banning** of SPOs who misbehave during DKG or signing rounds. A banned SPO retains their Membership Token and deposit but is excluded from participating in roster formation for a time-limited period.

Exponential timeout: Each successive ban doubles the exclusion duration. For example, a first ban may last 1 epoch, a second ban 2 epochs, a fourth 4 epochs, and so on. This escalating penalty discourages repeated misbehavior while allowing recovery from occasional failures.

Ban initiation: The current roster initiates a ban via FROST group signature.

Ban expiry: Once the ban period elapses, the SPO automatically becomes eligible for roster participation again without needing to re-register.

> **Note**: The exact behavior of the ban list is still under discussion and may be refined in future iterations.

8. Security Properties

- **Cold key minimization**: The cold key is used only twice—once for registration, once for revocation (if needed). All other protocol operations use `bifrost_id_sk`.
- **Air-gapped signing**: Both registration and revocation messages can be constructed offline and signed on an air-gapped machine.
- **Sybil resistance**: One membership token per `pool_id` enforced by minting policy.
- **Economic security**: Security deposits create financial accountability for misbehavior.
- **No expiration**: Membership tokens remain valid indefinitely until explicitly revoked.

Distributed Key Generation (DKG)

1. Overview The FROST Distributed Key Generation (DKG) process runs **entirely off-chain** using SPOs' `bifrost_url` endpoints. Two separate DKGs are run each epoch, producing group public keys Y_{67} and Y_{51} with thresholds ensuring any signing subset controls ≥67% and ≥51% of delegated stake respectively. Each DKG also produces individual signing shares s_i for each participant. Upon successful completion, the **current roster** constructs and signs a Treasury Movement transaction that moves the treasury to the new Taproot address derived from Y_{51}, Y_{67}, and $Y_{federation}$ (see **Taproot address construction**), and posts the signed transaction to Cardano at `treasury_movement.ak` for watchtowers to relay to the source blockchain. No DKG result is posted on Cardano.

Prerequisite: SPOs must complete SPO Registration (see previous section) before participating in DKG.

2. Epoch Binding Each DKG instance is bound to a Cardano epoch. The candidate set is determined by the on-chain registry linked-list state at the end of the previous epoch, ensuring all SPOs have the same view of registered participants.

3. Threshold Calculation The threshold `t` is computed to guarantee that **any** subset of `t` signers controls stake above the security threshold. Since the worst case is the `t` SPOs with the smallest stakes, we define:

```
t = min { k : combined_stake(bottom k SPOs by stake) > security_threshold }
```

Where: - `security_threshold` is a protocol parameter (e.g., 51% of total delegated stake among Bifrost SPOs). - SPOs are ranked by their delegated stake at the epoch boundary. - `t` is the minimum number of SPOs such that even the weakest `t` SPOs exceed the threshold.

This ensures that **any** subset of `t` signers collectively controls sufficient stake to authorize bridge operations, regardless of which specific SPOs participate in a signing session.

4. Candidate Set and Ordering

4.1 Candidate Enumeration All SPOs with valid Bifrost Membership Tokens (present in the registry linked-list) are candidates for the DKG.

4.2 Canonical Ordering Candidates are ordered **lexicographically by** `bifrost_id_pk` (32-byte comparison). Each participant is assigned an index $i = 1..n$ based on their position in this ordering.

4.3 Candidate Information For each candidate P_i, the following information is retrieved: - `pool_id` — from Membership UTxO. - `bifrost_id_pk` — from Membership UTxO datum. - `bifrost_url` — from Membership UTxO datum. - `delegated_stake` — queried from Cardano ledger state.

5. Round 0: Initialization Each SPO P_i performs the following initialization steps:

1. Determine the current epoch.
2. Retrieve the registry linked-list state from the end of the previous epoch.
3. Enumerate all candidates from the linked-list.
4. Query delegated stake for each candidate.
5. Compute threshold t as described in Section 3.
6. Order candidates lexicographically by `bifrost_id_pk` and assign indices.
7. Verify own participation (own `pool_id` is in the candidate set).

6. Round 1: Commitments and Proofs of Knowledge Each SPO P_i performs the following steps per FROST specification [2]:

1. Construct a random polynomial $f_i(x)$ of degree $t - 1$ over the Secp256k1 scalar field.
2. Compute proof of knowledge σ_i of the degree-zero coefficient a_{i0}.
3. Compute public commitment $C_i = [\varphi_{i0}, ..., \varphi_{i(t-1)}]$ where $\varphi_{ij} = a_{ij} \cdot G$.

6.1 Round 1 Payload Each P_i publishes their Round 1 data at:

`<bifrost_url>/dkg/<epoch>/round1/<pool_id>.json`

Payload structure:

```
{
  "commitment": ["<hex, 33 bytes>", ...],
  "sigma_i": "<hex, 64 bytes>"
}
```

Where: - `commitment` is an array of t compressed Secp256k1 points (33 bytes each). - `sigma_i` is the Schnorr proof of knowledge (challenge || response, 64 bytes).

6.2 Round 1 Verification Each P_i fetches Round 1 payloads from all other participants and verifies that σ_i is a valid proof of knowledge for φ_{l0}.

If verification fails for any participant P_l, the process proceeds to **Misbehavior Handling** (Section 9).

7. Round 2: Secret Share Distribution Each SPO P_i computes and distributes secret shares to all other participants.

7.1 Share Computation For each participant P_l (where $l \neq i$), compute the secret share $(l, f_i(l))$.

7.2 Share Encryption For each recipient P_l:

1. Generate ephemeral Secp256k1 keypair (e_i, E_i).
2. Compute shared secret: `ss = ECDH(e_i, bifrost_id_pk_l)`.
3. Derive symmetric key: `k = HKDF(ss, info = "bifrost-dkg-share")`.
4. Encrypt share: `ciphertext = f_i(l) XOR k` (32 bytes).

The share is a 32-byte Secp256k1 scalar, encrypted with the derived key.

7.3 Round 2 Payload Each P_i publishes their Round 2 data at:

`<bifrost_url>/dkg/<epoch>/round2/<pool_id>.json`

Payload structure:

```
{
  "shares": [
    {
      "recipient_pool_id": "<hex, 28 bytes>",
      "ephemeral_pk": "<hex, 33 bytes>",
      "ciphertext": "<hex, 32 bytes>"
    }
  ]
}
```

Where: - `recipient_pool_id` identifies the intended recipient. - `ephemeral_pk` is the compressed Secp256k1 ephemeral public key E_i. - `ciphertext` is the XOR-encrypted share. - The `shares` array contains $n - 1$ entries (one per other participant).

7.4 Round 2 Decryption and Verification Each recipient P_l:

1. Fetch Round 2 payload from each sender P_i.
2. Find the entry where `recipient_pool_id == pool_id_l`.
3. Compute shared secret: `ss = ECDH(bifrost_id_sk_l, ephemeral_pk)`.
4. Derive key `k = HKDF(ss, info = "bifrost-dkg-share")` and decrypt: `f_i(l) = ciphertext XOR k`.
5. Verify the share against sender's Round 1 commitment:

   $f_i(l) \cdot G = \sum_{j=0}^{t-1}(l^j \cdot \varphi_{ij})$

If verification fails for any share from P_i, the process proceeds to **Misbehavior Handling** (Section 9).

8. Finalization Upon successful verification of all shares, each P_i:

1. Computes their long-lived private signing share: $s_i = \sum_{l=1}^{n} f_l(i)$
2. Computes their public verification share: $Y_i = s_i \cdot G$
3. Computes the group public key: $Y = \sum_{l=1}^{n} \varphi_{l0}$

All participants arrive at the same group public key Y.

The above steps are run **twice** — once with a threshold t_{67} (producing Y_{67}) and once with t_{51} (producing Y_{51}). The two DKGs can run concurrently with the same candidate set.

4. Derives the Bitcoin Treasury Taproot address from Y_{51}, Y_{67}, and $Y_{federation}$ (see **Taproot address construction**).
5. The **current roster** publishes the new group public keys Y_{67} and Y_{51} on Cardano at `treasury.ak`, authenticated by a FROST group signature from the current roster. This makes the new Treasury address publicly verifiable on-chain, allowing depositors to look up the correct Treasury keys and derive the Treasury and peg-in Taproot addresses.

9. Misbehavior Handling If any participant P_m misbehaves (invalid proof of knowledge in Round 1, or invalid share in Round 2), the current roster **bans** them (Section 7.2) with an exponential timeout and restarts DKG.

9.1 DKG Restart After the ban transaction is confirmed on Cardano:

1. Banned SPOs are excluded from the candidate set for this epoch's DKG.
2. Threshold t is recomputed with the reduced candidate set.
3. DKG restarts from Round 0.

Note: Banning temporarily excludes the SPO with an exponentially increasing timeout, allowing them to rejoin after the ban expires.

10. Treasury Handoff Upon successful DKG completion and publication of the new Treasury public keys Y_{67} and Y_{51} to `treasury.ak`:

1. The **new roster** derives the Bitcoin Treasury Taproot address from Y_{51}, Y_{67}, and $Y_{federation}$ (see **Taproot address construction**).
2. The **current roster** reads all confirmed PegInRequest UTxOs and pending PegOut UTxOs from Cardano.
3. The **current roster** attempts to construct and sign a full Treasury Movement transaction (peg-ins + peg-outs + treasury move to new address) using the tiered signing process (see **Spending paths and Treasury Movement variants**):
   - First, attempt to collect 67% partial signatures (Y_{67}) — proves the stronger security threshold on Bitcoin (script path on treasury).
   - If 67% quorum is not reached, attempt to collect 51% partial signatures (Y_{51}) — main line, cheapest (key path on all inputs).
   - If neither quorum is reached within the timeout, the federation signs using $Y_{federation}$ (script path with timelock).
   - If the resulting transaction would be too large, it is split into multiple transactions.
4. The signed transaction is posted to Cardano at `treasury_movement.ak`.
5. Watchtowers pick up the signed transaction from Cardano and broadcast it to the Bitcoin network.

Once the Treasury Movement transaction is confirmed on Bitcoin, the epoch transition is complete. The new roster now controls the treasury. Anyone can then complete pending peg-outs on Cardano using Binocular inclusion proofs. Pending peg-ins can also be completed — all quorum levels sweep peg-in UTxOs.

11. Security Properties

- **Off-chain execution**: No DKG data is posted on Cardano; only the signed Treasury Movement transaction (posted to `treasury_movement.ak`) and the resulting source blockchain transaction are publicly visible.
- **Threshold security**: Any t signers control stake above the security threshold.
- **Misbehavior accountability**: Fraudulent SPOs can be identified and excluded.
- **Current roster authority**: Only the current roster can authorize exclusions, preventing new roster self-dealing.
- **Replay resistance**: Each DKG is bound to a unique epoch number.
- **Single curve**: Using Secp256k1 throughout eliminates curve conversion complexity.

Group signing

In what follows we summarize the *preprocess* and signing stages according to the FROST documentation [2], closely following their notation, and emphasizing special considerations relevant to SPO-based FROST groups.

Preprocess Each SPO P_i in the roster performs this stage prior to signing. 1. Samples random single-use nonces (d_{ij}, e_{ij}). 2. Derives commitment shares (D_{ij}, E_{ij}). 3. Stores $((d_{ij}, D_{ij}), (e_{ij}, E_{ij}))$ for later use in signing operations. 4. With L_i the list of (D_{ij}, E_{ij}), publishes (i, L_i) as datum attached to UTxO with participation token.

Signing mechanism

Each SPO P_i in the subset participating in signing performs these steps. 1. Receives message m to be signed and queries from blockchain the list B of triads (i, D_i, E_i) corresponding to SPOs in the subset. 2. Each P_i then computes the set of binding values, the group commitment R and the challenge. 3. Each P_i computes their response (signing share) z_i using their long-lived secret share s_i. 4. Each P_i verifies the validity of each response z_i, identifying and reporting misbehaving participants. If a misbehaving participant exists, process is aborted; otherwise continue. 5. Each P_i can compute the group's response (the sum of z_i's), arriving to the same signature $\sigma = (R, z)$, which they can publish along with message m.

SPOs communication

TODO

Watchtowers

Watchtower Architecture

Watchtowers are permissionless participants who maintain Bitcoin blockchain state on Cardano. They serve as the critical link between the Bitcoin and Cardano networks, ensuring that Bifrost has accurate, up-to-date information about the Bitcoin blockchain. Watchtowers use Binocular, a technology stack previously created and now improved on Bifrost.

Key Design Principles:

- **Permissionless Participation**: Anyone can become a watchtower at any time without registration, bonding, or approval. This ensures the system cannot be censored or controlled by a small group.
- **Competitive Model**: Multiple watchtowers compete to submit the most accurate chain of blocks. If one watchtower submits invalid or stale data, others can immediately challenge with the correct chain.
- **Economic Incentives**: Watchtowers are rewarded for posting valid blocks, creating a natural incentive for honest and timely participation.

Core Watchtower Responsibilities

1. **Monitor Bitcoin Network**: Watchtowers continuously track the Bitcoin blockchain for new blocks as they are mined.
2. **Submit Block Headers**: When new Bitcoin blocks are found, watchtowers submit the 80-byte block headers to Binocular Oracle smart contract on Cardano. These headers contain all information needed to verify Bitcoin consensus rules.

3. **Compete for Accuracy**: Multiple watchtowers naturally compete to submit the most accurate chain. If a watchtower submits headers from an invalid or weaker fork, other watchtowers can challenge by submitting the correct chain with higher cumulative proof-of-work.
4. **Maintain Oracle Liveness**: Watchtowers ensure the Oracle never becomes stale by continuously updating it with the latest Bitcoin state. This is essential for timely peg-in and peg-out processing.

Bifrost-Specific Watchtower Duties

Beyond maintaining general Bitcoin state, watchtowers perform specialized duties for the Bifrost bridge. The architecture has the following key constraints: SPO programs have access to Cardano chain state but not to Bitcoin chain state; watchtowers have access to Bitcoin chain state but not to SPO programs or SPO private keys. Cardano serves as the shared data layer between both parties.

Peg-in Detection and Posting

- Monitor the Bitcoin network for peg-in transactions by scanning for OP_RETURN outputs with the `"BFR"` prefix.
- Each peg-in transaction sends BTC to a unique Taproot address (Y_{51} key path for SPO sweep, $Y_{federation}$ script leaf for federation emergency sweep, or depositor timeout script leaf for self-refund; see **Taproot address construction**) and includes an OP_RETURN output: `"BFR" || depositor_pubkey_hash (20 bytes)` (23 bytes total). The depositor_pubkey_hash is HASH160 of the depositor's Bitcoin x-only public key and is needed by SPOs to reconstruct the Taproot tweak for key-path signing. Because each peg-in goes to a unique Taproot address (derived from the depositor's pubkey hash), watchtowers cannot track peg-ins by address alone — the OP_RETURN metadata is what makes them identifiable.
- Once a peg-in transaction reaches the required confirmation threshold (100 Bitcoin blocks plus 200 minutes of Binocular challenge period), watchtowers create a PegInRequest UTxO on Cardano (peg_in.ak) by:
  - Minting a PegInRequest NFT.
  - Providing a transaction inclusion proof consisting of: the raw Bitcoin transaction data, a Merkle proof linking the transaction to the block's Merkle root, and an inclusion proof of the confirmed block in the Binocular Oracle.
  - Setting the datum with: the raw Bitcoin peg-in transaction bytes and the creator's Cardano pubkey hash (for PegInRequest closure authorization).
- The on-chain `peg_in.ak` validator verifies the Binocular inclusion proof and confirmation depth (100 Bitcoin blocks + challenge period) but does not parse the Bitcoin transaction. SPO programs parse the raw transaction off-chain to extract deposit data (txid, vout, amount, depositor pubkey hash from OP_RETURN, Taproot output key Q) and validate it before including the peg-in in the Treasury Movement transaction. The raw peg-in transaction is parsed on-chain only at mint time (by `bridged_asset.ak`) to extract the depositor_pubkey_hash and deposit amount. Taproot address correctness is **not** verified on-chain (Plutus V3 lacks secp256k1 point arithmetic builtins); instead, SPOs verify off-chain (see **Taproot address verification**).

Treasury Movement Relay

- Monitor Cardano's treasury_movement.ak for new signed Bitcoin transactions posted by SPOs.
- Pick up the serialized signed Bitcoin transaction from the UTxO datum.
- Broadcast the transaction to the Bitcoin network.
- This is a permissionless action: any watchtower (or any user) can relay the transaction.

Peg-out Completion (Optional)

- Once the Treasury Movement transaction is confirmed on Bitcoin, watchtowers can complete peg-outs on Cardano by providing Binocular inclusion proofs. However, this is not exclusive to watchtowers — anyone can perform peg-out completion with the right proofs, ensuring censorship resistance.
- For peg-out completion: provide a Binocular inclusion proof showing the Treasury Movement transaction paid the correct amount to the correct Bitcoin address, burn the locked fBTC and the peg-out NFT, and return the MIN_ADA to the withdrawer.
- Peg-in completion (minting fBTC) is performed by the depositor directly, not by watchtowers — the depositor must provide their Bitcoin x-only public key and a Schnorr signature to authorize minting to their chosen Cardano address (see **bridged_asset.ak**).

Anomaly Detection

- Continuously verify that Treasury BTC balance matches or exceeds circulating fBTC supply.
- Alert the system if invariants are violated.
- Trigger failover mechanisms if SPO signing stalls or quorum is lost.

Binocular Oracle

The Binocular Oracle is the on-chain component that stores and validates Bitcoin blockchain state on Cardano. It provides trustless verification without requiring trust in any external party. For complete technical details, see the Binocular Whitepaper [1].

Bitcoin Consensus Validation The Oracle validates all Bitcoin consensus rules directly on-chain:

- Proof-of-Work verification (block hash meets difficulty target)
- Difficulty adjustment validation (every 2016 blocks)
- Timestamp constraints (greater than median-time-past, less than 2 hours in future)
- Chain continuity (each block references valid parent)

Fork Management The Oracle maintains a tree of competing Bitcoin forks and automatically selects the canonical chain based on cumulative chainwork (total proof-of-work). This mirrors exactly how Bitcoin Core selects the best chain, ensuring the Oracle always reflects Bitcoin's true state.

Confirmation Tracking Blocks progress through multiple stages:

- Initially added to the forks tree when submitted
- Tracked for confirmation depth (distance from chain tip)
- Only blocks with 100+ confirmations are considered for finality

Transaction Inclusion Proofs For Bifrost operations, the Oracle provides data for Watchtowers to construct proofs that:

- Prove a specific transaction exists within a confirmed block
- Prove the block is part of the confirmed chain
- Enable trustless verification of peg-in deposits and peg-out completions

Challenge Period Mechanism

To prevent pre-computed attacks and ensure security, blocks are not immediately finalized when submitted:

1. **Submission**: A watchtower submits new Bitcoin block headers to the Oracle
2. **Challenge Window**: A 200-minute window opens during which any other watchtower can submit a competing fork with higher chainwork
3. **Resolution**: The Oracle automatically selects the chain with the highest cumulative proof-of-work
4. **Finalization**: After the challenge period expires and the block has 100+ confirmations, it becomes "confirmed" and can be used for peg-in proofs

This mechanism ensures that even if a malicious watchtower pre-computes a short fork, honest watchtowers have ample time to submit the correct chain.

Security: 1-Honest-Watchtower Assumption

Bifrost's watchtower design relies on a minimal trust assumption: only one honest watchtower needs to exist for the system to function correctly.

Why This Works:

- If all active watchtowers collude to censor or submit invalid data, any user can spin up their own watchtower
- The permissionless design means no one can prevent new watchtowers from joining
- Honest watchtowers are economically incentivized to challenge invalid submissions

Censorship Resistance:

- A user wanting to peg-in or peg-out can always become a watchtower themselves
- They can then submit the necessary Bitcoin blocks and proofs for their own transactions
- This ensures Bifrost remains operational even in adversarial conditions

References

[1] Nemish, Alexander. "Binocular: A Trustless Bitcoin Oracle for Cardano." 2025. https://github.com/lantr-io/binocular/blob/main/pdfs/Whitepaper.pdf

[2] Komlo, C. and Goldberg, I. "FROST: Flexible Round-Optimized Schnorr Threshold Signatures." RFC 9591, IETF, 2024. https://datatracker.ietf.org/doc/rfc9591/

[3] Wuille, P. et al. "BIP340: Schnorr Signatures for secp256k1." Bitcoin Improvement Proposal, 2020. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

[4] Wuille, P. et al. "BIP341: Taproot: SegWit version 1 spending rules." Bitcoin Improvement Proposal, 2020. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki

[5] *Bifrost On-Chain Validators* (Aiken): https://github.com/FluidTokens/ft-bifrost-bridge/tree/main/onchain/va